

AI LAB WEEK 3

1. A* — Misplaced Tiles Heuristic

```
import heapq

class PuzzleNode:

    def __init__(self, state, g=0, h=0, parent=None):
        self.state = state
        self.g = g
        self.h = h
        self.f = g + h
        self.parent = parent

    def __lt__(self, other):
        return self.f < other.f

    def get_blank_pos(self):
        for r in range(3):
            for c in range(3):
                if self.state[r][c] == 0:
                    return r, c

    def generate_children(self):
        children = []
        r, c = self.get_blank_pos()
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < 3 and 0 <= nc < 3:
                new_state = [list(row) for row in self.state]
                new_state[r][c], new_state[nr][nc] = (
                    new_state[nr][nc],
                    new_state[r][c],
                )
                tuple_state = tuple(tuple(row) for row in new_state)
                children.append(
                    PuzzleNode(tuple_state, g=self.g + 1, parent=self)
                )
```

```
    return children
```

```
def heuristic_misplaced(state, goal):
```

```
    misplaced = 0
```

```
    for r in range(3):
```

```
        for c in range(3):
```

```
            val = state[r][c]
```

```
            if val != 0 and val != goal[r][c]:
```

```
                misplaced += 1
```

```
    return misplaced
```

```
def a_star(start_state, goal_state):
```

```
    start_node = PuzzleNode(
```

```
        start_state, g=0, h=heuristic_misplaced(start_state, goal_state)
```

```
    )
```

```
    open_set = []
```

```
    heapq.heappush(open_set, start_node)
```

```
    closed_set = set()
```

```
    nodes_expanded = 0
```

```
    while open_set:
```

```
        current = heapq.heappop(open_set)
```

```
        nodes_expanded += 1
```

```
        if current.state == goal_state:
```

```
            path = []
```

```
            curr = current
```

```
            while curr:
```

```
                path.append(curr.state)
```

```
                curr = curr.parent
```

```
            return path[::-1], nodes_expanded
```

```
        closed_set.add(current.state)
```

```
        for child in current.generate_children():
```

```
            if child.state in closed_set:
```

```
                continue
```

```
            child.h = heuristic_misplaced(child.state, goal_state)
```

```
            child.f = child.g + child.h
```

```
            heapq.heappush(open_set, child)
```

```
    return None, nodes_expanded
```

```
if __name__ == "__main__":
```

```
goal = ((1, 2, 3), (8, 0, 4), (7, 6, 5))
```

```
start = ((2, 8, 3), (1, 6, 4), (7, 0, 5))
```

```
print("A* SEARCH USING MISPLACED TILES HEURISTIC")
```

```
print("-----")
```

```
path, nodes_expanded = a_star(start, goal)
```

```
print("Steps to solve:", len(path) - 1 if path else "No solution")
```

```
print("Nodes expanded:", nodes_expanded)
```

```
print("\nStep-by-step solution:\n")
```

```
if path:
```

```
    for i, state in enumerate(path):
```

```
        print(f"Step {i}:")
```

```
        for row in state:
```

```
            print(list(row))
```

```
        print()
```

```
A* SEARCH USING MISPLACED TILES HEURISTIC
-----
Steps to solve: 5
Nodes expanded: 7

Step-by-step solution:

Step 0:
[2, 8, 3]
[1, 6, 4]
[7, 0, 5]

Step 1:
[2, 8, 3]
[1, 0, 4]
[7, 6, 5]

Step 2:
[2, 0, 3]
[1, 8, 4]
[7, 6, 5]

Step 3:
[0, 2, 3]
[1, 8, 4]
[7, 6, 5]

Step 4:
[1, 2, 3]
[0, 8, 4]
[7, 6, 5]

Step 5:
[1, 2, 3]
[8, 0, 4]
[7, 6, 5]
```

